SymPy Bug Report

1 Bug 19: `rsolve` returns a solution that violates supplied initial conditions

1.1 Minimal Reproducer

```
from sympy import Function, rsolve, symbols

n = symbols("n", integer=True)
y = Function("y")

rec = y(n + 2) - (n + 2)*y(n + 1) + y(n)
sol = rsolve(rec, y(n), {y(0): 1, y(1): 1})

print("rsolve output:", sol)
print("returned y(0):", sol.subs(n, 0))
print("returned y(1):", sol.subs(n, 1))
```

1.2 Observed Behavior

```
rsolve output: 0
returned y(0): 0
returned y(1): 0
manual first terms: [1, 1, 1, 2, 7, 33]
```

SymPy returns the zero sequence. Direct substitution into that returned expression gives $y(0) = 0$ and $y(1) = 0$, even though the call explicitly supplied $y(0) = 1$ and $y(1) = 1$. The result is therefore not merely an incomplete closed form or a different representation of the intended sequence; it violates the constraints passed to the public API.

1.3 Expected Behavior

The solver should return a sequence satisfying the recurrence

$$y(n+2) - (n+2)y(n+1) + y(n) = 0$$

and the initial conditions $y(0) = 1$, $y(1) = 1$, or else report that it cannot solve the recurrence with those initial conditions. It should never return the zero sequence for this call.

1.4 Mathematical Explanation

The recurrence is equivalent to

$$y(n+2) = (n+2)y(n+1) - y(n).$$

Starting from the supplied initial values gives

$$\begin{aligned} y(0) &= 1, \\ y(1) &= 1, \\ y(2) &= 2y(1) - y(0) = 1, \\ y(3) &= 3y(2) - y(1) = 2, \\ y(4) &= 4y(3) - y(2) = 7, \\ y(5) &= 5y(4) - y(3) = 33. \end{aligned}$$

Thus a valid solution with these initial conditions begins

$$1,\ 1,\ 1,\ 2,\ 7,\ 33, \ldots.$$

The zero sequence does satisfy the homogeneous recurrence, but it satisfies $y(0) = 0$ and $y(1) = 0$, not the supplied nonzero initial conditions. Hence SymPy's answer is a solution of the recurrence alone, not a solution of the initial-value problem requested by the user.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant implementation as `sympy/solvers/recurr.py`, in the public `rsolve` function around lines 816–841. The traced call path is `sympy.rsolve(...)` to coefficient extraction, then to `rsolve_hyper(coeffs, 0, n, symbols=True)` near line 806, followed by the initial-condition handling in `rsolve`. For the representative recurrence, the traced internal call `rsolve_hyper([1, -(n + 2), 1], 0, n, symbols=True)` returns the constant-free candidate `0` with an empty list of arbitrary constants.

The problematic rule is that initial conditions are only applied when the candidate expression contains symbols for arbitrary constants:

```
if symbols and init is not None:
    ...
```

When `symbols` is empty, this block is skipped. In the failing case `solution` is already the constant-free expression `0`, so `rsolve` never forms or checks the equations `solution.subs(n, 0) - 1` and `solution.subs(n, 1) - 1`. It then returns the candidate unchanged. This source-level behavior causes the mathematical failure because a constant-free candidate can still contradict explicit initial conditions. The diagnosis supports a plausible fix direction: when initial conditions are provided, `rsolve` should validate constant-free solutions as well, returning `None` or the same unsolved initial-condition failure used elsewhere when any supplied condition is false.

1.6 Additional Failing Instances

The independent verification checks the family

$$y(n+2) - (n+a)y(n+1) + y(n) = 0, \qquad y(0) = 1, \quad y(1) = 1,$$

for $a = 2, 3, 4, 5, 6, 7$. In every listed case SymPy returns the zero sequence, so the returned values at $n = 0$ and $n = 1$ are both 0. The expected behavior is the same across the family: the returned expression must satisfy the two supplied initial values, or the solver should report that it cannot produce such a solution.

Input / Expression	**SymPy behavior**	**Expected behavior**
$a = 2:\ y(n+2) - (n+2)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0,\ y(1) = 0$	satisfy $y(0) = y(1) = 1$; forward terms $1, 1, 1, 2, 7, 33$
$a = 3:\ y(n+2) - (n+3)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0,\ y(1) = 0$	satisfy $y(0) = y(1) = 1$; forward terms $1, 1, 2, 7, 33, 191$
$a = 4:\ y(n+2) - (n+4)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0,\ y(1) = 0$	satisfy $y(0) = y(1) = 1$; forward terms $1, 1, 3, 14, 81, 553$
$a = 5:\ y(n+2) - (n+5)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0,\ y(1) = 0$	satisfy $y(0) = y(1) = 1$; forward terms $1, 1, 4, 23, 157, 1233$
$a = 6:\ y(n+2) - (n+6)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0,\ y(1) = 0$	satisfy $y(0) = y(1) = 1$; forward terms $1, 1, 5, 34, 267, 2369$
$a = 7:\ y(n+2) - (n+7)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0,\ y(1) = 0$	satisfy $y(0) = y(1) = 1$; forward terms $1, 1, 6, 47, 417, 4123$

1.7 Related Issues Assessment

A re-audit of the deduplication check identifies open public issues that report the same underlying `rsolve_hyper`-returns-0 failure family, in which `rsolve` silently returns the zero sequence for variable-coefficient / higher-order recurrences:

- sympy/sympy#27902, "rsolve: Gives 0 for higher order recurrences" (open) — the most direct match: its example `rsolve(y(n+2) - n*y(n+1) - y(n), y(n))` returns 0, the same second-order variable-coefficient symptom.
- sympy/sympy#17982, "Wrong result from rsolve" (open).
- sympy/sympy#11063, "Some wrong answers from rsolve" (open).

The deduplication verdict is therefore *family known, specific instance new*: the broad `rsolve`-returns-0 failure is publicly reported, while bug-019's specific lens — the constant-free 0 candidate causing the `if symbols and init is not None` initial-condition block to be skipped, so supplied conditions are never validated — is the new specific instance within that known family. The issue remains individually actionable because it has a compact reproducer, a localized source-level mechanism, and a direct regression test for the supported initial-condition API.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Function, S, rsolve, symbols


CASES = [2, 3, 4, 5, 6, 7]


@pytest.mark.parametrize("a", CASES)
def test_rsolve_constant_free_solution_satisfies_initial_conditions(a):
    n = symbols("n", integer=True)
    y = Function("y")
    rec = y(n + 2) - (n + a)*y(n + 1) + y(n)
    sol = rsolve(rec, y(n), {y(0): S.One, y(1): S.One})

    assert sol is not None
    assert sol.subs(n, 0) == S.One
    assert sol.subs(n, 1) == S.One
```